

Test Case Generation Assistant using GenAI

Overview

This document outlines the design, features, and architecture of a GenAI-powered assistant for automatic test case generation. It includes high-level and low-level design, system components, data flow, and the role of data science algorithms.

1. Features

Core Features

Requirement Parsing: Extracts structured data from functional and non-functional requirement documents.

Test Case Generation: Automatically creates manual/automated test cases, including edge cases.

Test Data Generation: Produces valid and invalid inputs for each test case.

Test Case Classification: Categorizes into unit, integration, system, and regression tests.

Traceability Mapping: Maps test cases back to original requirements.

Interactive Interface: Natural Language Query interface for users.

Feedback Loop: Users can rate and correct generated test cases to improve future outputs.

Version Tracking: Detects and adapts to changes in requirements.

Export Support: Outputs in CSV, Excel, or integrates with tools like Jira, TestRail.

API Integration: Connects with CI/CD pipelines and testing frameworks.

2. Requirement Document Structure

Business Context

Defines the overall business goal the software addresses.

Functional Requirements

FR1: The system shall allow user registration with email and password.

FR2: The system shall support password reset.

FR3: The system shall log failed login attempts.

Non-Functional Requirements

NFR1: Response time < 2s.

NFR2: 99.9% uptime.

User Stories & Use Cases

As a user, I want to register, so I can access the platform.

As an admin, I want to view login statistics.

3. High-Level Design (HLD)

Components

UI Interface: Web/CLI for user interaction.

NLP Engine: Parses and understands requirements.

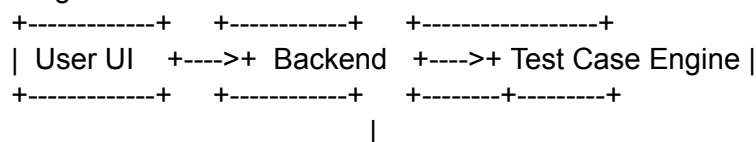
Test Case Generator: Core GenAI logic for generating test cases.

Traceability Mapper: Maps test cases to requirements.

Storage Layer: Stores documents, test cases, mappings.

Feedback Loop Engine: Learns from user input.

Diagram



v
+-----+
| Storage|
+-----+

4. Low-Level Design (LLD)

Modules

Requirement Parser

Uses spaCy/transformers to extract entities (e.g., user actions, conditions).

Converts text to structured JSON.

Test Case Generator

Uses GPT-4/GPT-4o with prompt templates.

Input: Requirement JSON

Output: Test case objects

Feedback Loop

Uses reinforcement learning or ranking models.

Applies corrections to fine-tune future results.

Data Flow

[Requirement Doc] --> [Parser] --> [Structured Req] --> [GenAI Model]
--> [Generated Test Cases]
--> [User Feedback] --> [Model Fine-tuning]

5. Test Case Schema (Manually Created)

Field

Description

Test Case ID

Unique identifier

Title

Short title of the test case

Description

Steps and scenario description

Preconditions

Setup required before execution

Steps

List of actions

Expected Results

Outcome after each step

Priority

High / Medium / Low

Requirement ID

Linked requirement

6. Data Science Algorithms

Algorithms Used

Transformer models (e.g., BERT, GPT-4o): Requirement understanding and test case generation.

NER (Named Entity Recognition): Extracts important terms.

Clustering (KMeans): Groups similar requirements.

Sequence Models (LSTM): Predicts logical step sequences.

RLHF (Reinforcement Learning with Human Feedback): Improves based on user reviews.

Use Cases

Predict test case templates

Suggest missing tests

Identify redundant/overlapping cases

Conclusion

This design provides a scalable, intelligent, and interactive assistant for test case generation. By combining LLMs, traditional data science, and user feedback, the system ensures high-quality and maintainable test coverage.